You can see how poorly it works in figure 2.16.

```
d3.select("svg")
  .selectAll("rect")
  .data([14, 68, 24500, 430, 19, 1000, 5555])
  .enter()
  .append("rect")
  .attr("width", 10)
  .attr("height", function(d) {return d})
  .style("fill", "blue")
  .style("stroke", "red")
  .style("stroke-width", "1px")
  .style("opacity", .25)
  .attr("x", function(d,i) {return i * 10;})
  .attr("y", function(d) {return 100 - d;});
```

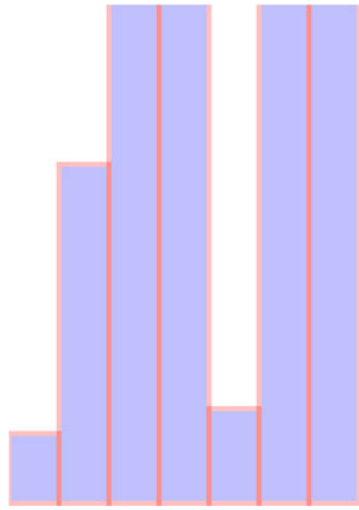


Figure 2.16 SVG shapes will continue to be drawn offscreen.

And it works no better if you set a y offset equal to the maximum:

```
d3.select("svg")
  .selectAll("rect")
  .data([14, 68, 24500, 430, 19, 1000, 5555])
  .enter()
  .append("rect")
  .attr("width", 10)
  .attr("height", function(d) {return d})
  .style("fill", "blue")
  .style("stroke", "red")
  .style("stroke-width", "1px")
  .style("opacity", .25)
  .attr("x", function(d,i) {return i * 10;})
  .attr("y", function(d) {return 24500 - d;});
```

There's no need to bother with a screenshot. It's just a single bar running vertically across your canvas. In this case, it's best to use D3's scaling functions to normalize the values for display. We'll use the relatively straightforward `d3.scale.linear()` for this bar chart. A D3 scale has two primary functions: `.domain()` and `.range()`, both of which expect arrays and which must have arrays of the same length to get the right results. The array in `.domain()` indicates the series of values being mapped to `.range()`, which will make more sense in practice. First, we make a scale for the y-axis:

```
var yScale = d3.scale.linear().domain([0,24500]).range([0,100]);
yScale(0);
yScale(100);
yScale(24000);
```

Returns 0

Returns 0.40816326530612246

Returns 97.95918367346938

As you can see, `yScale` now allows us to map the values in a way suitable for display. If we then use `yScale` to determine the height and y position of the rectangles, we end up with a bar chart that's more legible, as shown in figure 2.17.